

UNIDAD 2



HERENCIA, POLIMORFISMO Y CLASES ABSTRACTAS E INTERFACES



Objetivo

Comprender cómo reutilizar código y crear programas más organizados mediante **herencia, polimorfismo, clases abstractas e interfaces**.



Reutiliza
código



Organiza tu
programa



Más seguro y
mantenible



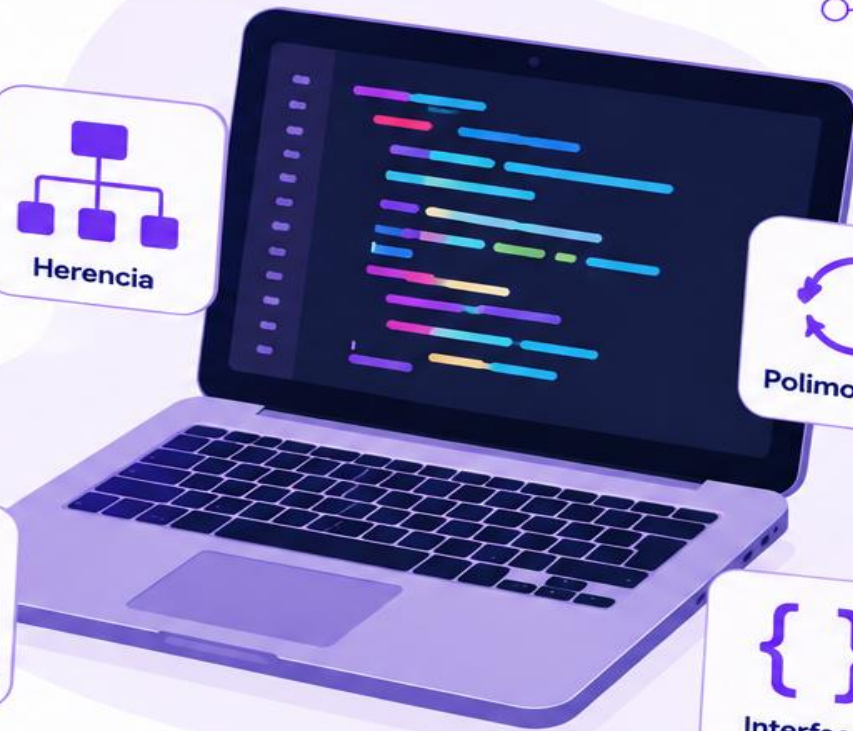
Mejores
aplicaciones



Herencia



Clases abstractas



Polimorfismo



Interfaces

Temas de la unidad

2.1 Herencia

Reutilización de código.
Relación padre e hijo.

2.2 Polimorfismo

Un mismo método
puede comportarse de
distintas formas.

2.3 Clases Abstractas e Interfaces

Definir comportamientos
comunes. Establecer reglas
para las clases.

2.1

Herencia



¿Qué es?

La herencia permite que una clase (hija) obtenga características y comportamientos de otra clase (padre).



¿Para qué sirve?

Para reutilizar código, evitar repetirlo y crear relaciones entre clases.



Ejemplo de la vida real

Clase Padre: Animal

Características:

- nombre
- edad

Acciones:

- comer()
- dormir()



Clase Hija: Perro

Características heredadas:

- nombre
- edad

Acciones heredadas:

- comer()
- dormir()

Además puede:

- ladrar()



Perro hereda todo lo de **Animal** y además tiene su propia acción: **ladrar()**.



Ejemplo en Java

```
class Animal {
    String nombre;
    int edad;

    void comer() {
        System.out.println("Comiendo");
    }

    void dormir() {
        System.out.println("Durmiendo");
    }
}

class Perro extends Animal {
    void ladrar() {
        System.out.println("Guau");
    }
}
```

Uso:

```
Perro miPerro = new Perro();
miPerro.comer(); // Comiendo
miPerro.dormir(); // Durmiendo
miPerro.ladrar(); // Guau
```

Salida en consola:

```
Comiendo
Durmiendo
Guau
```



Idea clave

La clase hija (**Perro**) hereda los atributos y métodos de la clase padre (**Animal**) y puede agregar nuevos comportamientos propios.

Animal
(Clase padre)



Perro
(Clase hija)

2.2

Polimorfismo



¿Qué es?

Significa "muchas formas".
Un mismo método puede generar resultados diferentes según el objeto que lo utilice.



¿Para qué sirve?

Permite escribir código más flexible, reutilizable y fácil de mantener.



Ejemplo de la vida real

Todos los animales hacen sonido, pero cada uno suena diferente.



Perro

Guau



Gato

Miau



Vaca

Muuu



Idea clave

El mismo método (sonido) se comporta de manera diferente según el objeto que lo usa.



Ejemplo en Java

```
class Animal {  
    void sonido() {  
        System.out.println("Sonido");  
    }  
}  
  
class Perro extends Animal {  
    @Override  
    void sonido() {  
        System.out.println("Guau");  
    }  
}  
  
class Gato extends Animal {  
    @Override  
    void sonido() {  
        System.out.println("Miau");  
    }  
}
```

Uso:

```
Animal a1 = new Perro();  
Animal a2 = new Gato();  
  
a1.sonido(); // Guau  
a2.sonido(); // Miau
```

Salida en consola:

```
Guau  
Miau
```



Polimorfismo = un mismo mensaje, diferentes respuestas.

Método
sonido()



Objeto
Perro

Responde
"Guau"



Objeto
Gato

Responde
"Miau"



Objeto
Vaca

Responde
"Muuu"



2.3

Clases Abstractas e Interfaces



Clases Abstractas

Una clase abstracta es una clase que no se puede instanciar directamente y sirve como base para otras clases. Puede tener métodos con y sin implementación.

Ejemplo

```
abstract class Animal {  
    String nombre;  
    Animal(String nombre) {  
        this.nombre = nombre;  
    }  
  
    // Método abstracto (sin implementación)  
    abstract void sonido();  
}
```



No se puede crear un objeto de **Animal**, pero otras clases pueden heredar de ella.



Interfaces

Una interfaz define un conjunto de reglas (métodos) que una clase debe cumplir. Solo tiene métodos sin implementación (por defecto son públicos).

Ejemplo

```
interface Volador {  
    void volar();  
}
```

Implementación

```
class Pajaro implements Volador {  
    @Override  
    public void volar() {  
        System.out.println("Estoy volando");  
    }  
}
```



Pajaro debe implementar el método **volar()** porque la interfaz **Volador** lo exige.



Idea clave



Clase abstracta

Sirve como plantilla base para otras clases. No se puede instanciar.



Interface

Define un contrato o regla que las clases deben cumplir.



Ambas

Ayudan a crear programas más organizados, flexibles y fáciles de mantener.

Herencia, Polimorfismo e Interfaces en acción

1 Clase padre

Clase base que será heredada.



```
class Animal {  
    void sonido() {  
        System.out.println  
            ("Sonido");  
    }  
}
```

2 Herencia

La clase hija hereda de la clase padre.



```
class Perro extends Animal {  
    @Override  
    void sonido() {  
        System.out.println  
            ("Guau");  
    }  
}
```

3 Interface

Define una regla que las clases deben cumplir.



```
interface Corredor {  
    void correr();  
}
```

4 Implementación

La clase Perro cumple la regla de Corredor.



```
class Perro extends Animal  
    implements Corredor {  
    @Override  
    void sonido() {  
        System.out.println  
            ("Guau");  
    }  
    @Override  
    public void correr() {  
        System.out.println  
            ("Corriendo");  
    }  
}
```

5 Uso (Polimorfismo)

Un mismo método (sonido) se comporta diferente según el objeto.



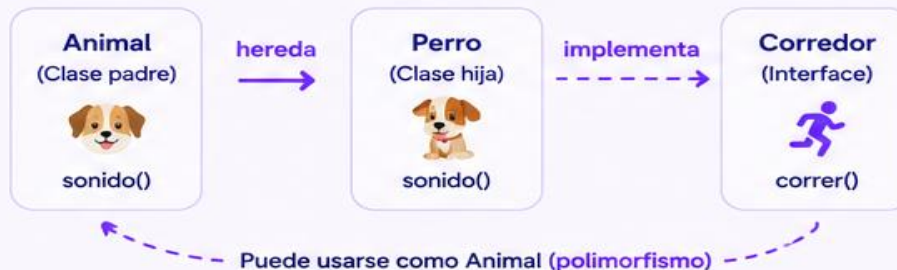
```
public class Main {  
    public static void main  
        (String[] args) {  
        Animal a1 = new Perro();  
        Animal a2 = new Animal();  
  
        a1.sonido(); // Guau  
        a2.sonido(); // Sonido  
  
        ((Corredor)a1).correr();  
        // Corriendo  
    }  
}
```



¿Qué sucede?

- ✓ Perro hereda de Animal (obtiene sonido).
- ✓ Perro implementa Corredor (debe correr).
- ✓ El método sonido() cambia según el objeto (polimorfismo).

Resumen visual



Idea clave

- ✓ Herencia: reutiliza y organiza código.
- ✓ Polimorfismo: un método, muchas formas.
- ✓ Clases abstractas: plantilla base.
- ✓ Interfaces: reglas que deben cumplirse.
- ✓ Juntos: crean programas flexibles, escalables y fáciles de mantener.



GRACIAS

— POR TU ATENCIÓN —

...

